
Vietnamese – English Speech Translation

Nguyen Ngan Huong

13 June 2026

Contents

1	Introduction	2
2	Baseline Evaluation	2
2.1	System Configuration	2
2.1.1	Base Model	2
2.1.2	Generation Configuration	3
2.1.3	Training Hyperparameters	3
2.1.4	Parameter-Efficient Fine-Tuning (LoRA)	3
2.1.5	Training Data	4
2.2	Baseline Results	4
3	Experiments	4
3.1	A smaller learning rate dramatically improved test set generalization but destroyed validation set generation	4
3.2	Investigating the Validation–Leaderboard Mismatch	6
3.2.1	Generation settings	6
3.2.2	Lowercase settings	7
3.2.3	Reference Mismatch	7
3.3	Decoding Exploration with Test-Style Validation	8
3.4	FLEURS Dataset Influence	9
3.5	Model Capacity	10
4	Conclusion	11

Abstract

The goal of this project was to improve the BLEU score of a Vietnamese-to-English speech translation system based on the provided Whisper-LoRA baseline. Beyond merely enhancing baseline performance, I investigated strategies and evaluation behavior to identify factors that meaningfully affect performance and draw insights about model improvement in speech processing.

1. Introduction

Automatic speech translation (AST) is the task of converting spoken utterances in one language directly into text in another, without producing an intermediate transcript. This project addresses the Vietnamese-to-English direction.

The test dataset consists of 2,481 audio clips totalling approximately 9.1 hours of 16 kHz mono `.wav` recordings drawn from four Vietnamese YouTube channels covering current-affairs and cultural commentary. Because the audio originates from chunked long-form video, it exhibits natural prosodic variation, background noise, and domain-specific vocabulary.

Submissions are evaluated with corpus-level BLEU-4 computed via `sacrebleu.corpus_bleu` with lowercase tokenization, returning a score on a 0–100 scale; higher values indicate closer agreement with the human reference translations. The baseline system provided fine-tunes `openai/whisper-small` on the FLEURS `vi_vn↔en_us` parallel corpus using Low-Rank Adaptation (LoRA), a parameter-efficient training technique that inserts small trainable adapter matrices into a frozen pretrained model.

2. Baseline Evaluation

2.1 System Configuration

All experiments run on a single NVIDIA Tesla T4 GPU inside the Kaggle notebook environment. The baseline pipeline consists of five stages:

- Audio loading
- Log-mel feature extraction
- LoRA fine-tuning
- Adapter merging
- Translation on test clips

2.1.1 Base Model

The pretrained model is `openai/whisper-small`, which is loaded in `bfloat16` half-precision with the scaled dot-product attention (`sdpa`) implementation to reduce memory consumption.

- Model: `openai/whisper-small`
- Language: `language="vi"` (Vietnamese audio input)
- Task: `task="translate"` (direct speech-to-English-text output)

2.1.2 Generation Configuration

Before fine-tuning, the baseline clears Whisper’s built-in decoding defaults to prevent them from conflicting with arguments passed at inference time. Table 1 lists every field that is reset.

Table 1. Generation configuration: fields cleared before fine-tuning

Parameter	Value	Effect
<code>generation_config.num_beams</code>	1	Greedy decoding; consistent with <code>INFER_BEAMS = 1</code> at inference.
<code>generation_config.no_repeat_ngram_size</code>	0	Disables bigram blocking during training evaluation. Re-enabled to 2 at inference.
<code>generation_config.suppress_tokens</code>	[]	Removes the per-step token suppression list.
<code>generation_config.forced_decoder_ids</code>	None	Clears forced prefix.
<code>config.suppress_tokens</code>	None	Prevents a duplicate <code>SuppressTokensLogitsProcessor</code> at <code>generate()</code> time.
<code>config.begin_suppress_tokens</code>	None	Disables first-step silence suppression.
<code>config.forced_decoder_ids</code>	None	Ensures no residual forced prefix in the Trainer’s internal <code>generate()</code> call.

2.1.3 Training Hyperparameters

Table 2. Training hyperparameters

Variable	Description	Value
<code>NUM_EPOCHS</code>	Number of training epochs	3
<code>TRAIN_BATCH</code>	Per-device train batch size	8
<code>GRAD_ACCUM</code>	Gradient accumulation steps	4
<code>TRAIN_BATCH × GRAD_ACCUM</code>	Effective batch size	32
<code>LEARNING_RATE</code>	Learning rate	1×10^{-3}
<code>INFER_BATCH</code>	Clips translated per batch	8
<code>INFER_BEAMS</code>	Beam width at decoding	1
<code>SAMPLING_RATE</code>	Audio sampling rate (Hz)	16,000

2.1.4 Parameter-Efficient Fine-Tuning (LoRA)

LoRA freezes the pretrained weights and trains only small adapter matrices injected into six linear projection layers, keeping GPU memory within T4 limits.

Table 3. LoRA adapter hyperparameters

Hyperparameter	Value
Rank (r)	8
Alpha (α)	16
Dropout	0.05
Bias	none
Target modules	q_proj, k_proj, v_proj, out_proj, fc1, fc2
Trainable parameters	3,244,032 (1.32% of total)
Total parameters	244,978,944

2.1.5 Training Data

The model is fine-tuned on FLEURS, pairing `vi_vn` audio with `en_us` transcriptions matched by a shared `id` field.

Table 4. FLEURS dataset splits

Split	Examples
Train	2,927
Validation	361
Test (FLEURS internal)	857

2.2 Baseline Results

Table 5. Baseline training results on FLEURS validation split

Epoch	Training Loss	Validation Loss	Validation BLEU
1	23.316	2.489	4.44
2	13.846	2.505	5.53
3	10.446	2.567	5.42

3. Experiments

3.1 A smaller learning rate dramatically improved test set generalization but destroyed validation set generation

Table 6. Baseline Result: `learning_rate=1e-3`

Epoch	Training loss	Validation loss	Validation BLEU
1	23.315625	2.489810	4.436899
2	13.845788	2.505435	5.529789
3	10.446332	2.567255	5.420454

Table 7. Experiment Result: `learning_rate=5e-5`

Epoch	Training Loss	Validation Loss	Validation BLEU
1	36.009028	3.966712	5.409163
2	26.511549	3.309103	6.695997
3	24.218071	3.220109	7.392296

Table 8. Experiment Result: `learning_rate=1e-5`

Epoch	Training Loss	Validation Loss	Validation BLEU
1	40.231944	4.949728	3.699035
2	36.739130	4.638587	4.020436
3	35.357337	4.582880	3.803702

Table 9. Learning rate ablation study.

Setting	Baseline	E2	E3
Learning Rate	1×10^{-3}	1×10^{-5}	5×10^{-5}
Epochs	3	3	3
Beam Size	1	1	1
no_repeat_ngram_size (test generation)	2	2	2
Training Loss	13.8458	36.7391	24.2181
Validation Loss	2.5054	4.6386	3.2201
Validation BLEU*	5.53	4.02	7.39
Public BLEU	6.86	12.56	9.05

*Validation BLEU corresponds to the best epoch selected by the notebook.

The baseline notebook used a learning rate of 1×10^{-3} . I hypothesized that this value might be too aggressive for LoRA fine-tuning on a relatively small speech translation dataset. To test this hypothesis, I first reduced the learning rate to 1×10^{-5} while keeping all other hyperparameters unchanged.

The result was surprising. Although validation BLEU decreased from 5.53 to 4.02, public BLEU increased substantially from 6.86 to 12.56. This observation suggested that local validation BLEU might not be a reliable indicator of leaderboard performance. However, since this conclusion was based on a single experiment, I conducted a second experiment using an intermediate learning rate of 5×10^{-5} to verify whether the discrepancy persisted.

The second experiment further strengthened this observation. The model trained with 5×10^{-5} achieved the highest validation BLEU (7.39) compared to the baseline (5.53) and the E2 model (4.02), but its public BLEU was only 9.05, substantially lower than the 12.56 achieved by the E2 model. Therefore, the model with the best validation BLEU did not produce the best leaderboard performance.

There are two possible explanations:

- First, the decoder of validation set generation is different from the test set generation. The validation generation uses internal function `generate()` from

`Seq2SeqTrainer` while the test generation uses the manual function `model.generate()` in the `translate_wavs()`. The notable difference is that the validation generation uses `base.generation_config.no_repeat_ngram_size = 0` and the test generation uses `no_repeat_ngram_size = 2`. Moreover, the `Trainer()` compared the predictions with tokenizer-roundtrip references, meaning references that had been tokenized and decoded again in the same style as the `Trainer` metric. Meanwhile, the scoring rule compared against raw references by using `sacrebleu.corpus_bleu(hyps, [refs], lowercase=True, tokenize="13a")`.

- Second, there is a distribution mismatch between the FLEURS dataset and the competition test set. Under this hypothesis, larger learning rates encourage stronger adaptation to the FLEURS training distribution. This adaptation improves local validation metrics with smaller training loss and validation loss, as observed with the 5×10^{-5} model, but may reduce generalization to the competition test set, resulting in lower public BLEU. In contrast, the 1×10^{-5} model learns more conservatively, resulting in lower validation BLEU but preserving more of Whisper’s pretrained speech translation capability. Consequently, it may generalize better to unseen data that differs from the FLEURS distribution and get better public BLEU score.

3.2 Investigating the Validation–Leaderboard Mismatch

3.2.1 Generation settings

The original validation BLEU was computed through the `Seq2SeqTrainer` evaluation pipeline, where `no_repeat_ngram_size=0`. In contrast, competition submissions were generated manually through `model.generate()` inside `translate_wavs()`, using `no_repeat_ngram_size=2`.

This mismatch was visible in the validation errors. When I printed the validation examples with the lowest sentence-level BLEU scores, many predictions contained severe repetition loops. For example:

Reference:

soon officers equipped with riot gear entered the yard and cornered the inmates with tear gas

Prediction:

After that, the police were equipped with anti-terrorism tools, anti-terrorism tools, anti-terrorism tools, anti-terrorism tools, anti-terrorism tools, anti-terrorism tools, ...

Reference:

108 plates of chhappan bhog in hinduism 56 different edible items likesweets fruits nuts dishes etc. which are offered to deity were servedto baba shyam

Prediction:

108 places to have fun and eat in the temple of the church

56 dishes

like sweet food, fruit, spices, dishes are served to the gods,
 theBuddha, the Buddha, the Buddha, the Buddha, the Buddha, the Buddha,
 the Buddha, the Buddha, the Buddha, the Buddha, the Buddha, the Buddha,
 the Buddha, ...

This error pattern suggested that removing the repetition constraint allowed the model to repeatedly generate high-confidence phrases instead of stopping or continuing the translation correctly. I therefore re-evaluated validation using `no_repeat_ngram_size=2`, matching the original test-generation setup.

Table 10. Effect of `no_repeat_ngram_size` on validation BLEU

Learning Rate	Validation BLEU ($n = 0$)	Validation BLEU ($n = 2$)
1×10^{-5}	4.02	6.62
5×10^{-5}	7.39	7.06

The result confirmed that decoding settings strongly affected validation BLEU. For the `learning_rate=1e-5` model, validation BLEU increased from 4.02 to 6.62. However, this correction did not fully resolve the validation–leaderboard mismatch: the `learning_rate=5e-5` model still had higher validation BLEU, while the `learning_rate=1e-5` model had higher public BLEU.

3.2.2 Lowercase settings

Some predictions differed from the references mainly in capitalization. Since BLEU is case-sensitive when `lowercase=False`, these surface-form differences reduce n-gram overlap even when the underlying words are the same. I therefore recomputed validation BLEU with `lowercase=True` while keeping `no_repeat_ngram_size=2`.

Table 11. Effect of lowercasing on validation BLEU after applying `no_repeat_ngram_size=2`

Learning Rate	Validation BLEU (lowercase=True)	Validation BLEU (lowercase=False)
1×10^{-5}	8.14	6.62
5×10^{-5}	8.74	7.06

Lowercasing made the local metric closer to the public evaluation protocol and substantially increased validation BLEU. However, the relative ranking between the two learning-rate models still did not match the leaderboard.

3.2.3 Reference Mismatch

I found that the reference text used by the Trainer metric was also different from the raw validation references. In the original `compute_metrics()` function, references were reconstructed from tokenized label IDs:

```
label_ids[label_ids == -100] = processor.tokenizer.pad_token_id
label_str = processor.tokenizer.batch_decode(
    label_ids,
    skip_special_tokens=True
```

)

Thus, the Trainer-reported BLEU compared predictions against tokenizer-decoded references, which constructed by tokenizing each raw reference with the Whisper tokenizer and decoding it back into text. Meanwhile, scoring rules directly compared predictions against the raw strings in `dataset["validation"]["translation"]`.

```
sacrebleu.corpus_bleu(hyps, [refs], lowercase=True, tokenize="13a")
```

To verify the effect of this difference, I compared the same validation predictions against two reference versions: raw validation references and tokenizer-roundtrip references.

Table 12. Effect of reference preprocessing on validation BLEU for the $LR=1 \times 10^{-5}$ model

Reference used for BLEU	Validation BLEU
Test-style validation	9.09
Trainer-reported validation	9.06

where `num_beams=3` and `no_repeat_ngram_size=3`

For the `learning_rate=1e-5` model, the raw-reference BLEU was 9.09, while the tokenizer-roundtrip reference BLEU was 9.06. Since the difference was very small, I concluded that reference preprocessing did not introduce a major evaluation mismatch in the final pipeline.

From this point onward, I used a separate function, `score_validation_like_test()`, which generates validation predictions through the same manual `model.generate()` path used for test inference and computes BLEU against raw validation references with `lowercase=True`.

3.3 Decoding Exploration with Test-Style Validation

After identifying the evaluation mismatch, I selected the `learning_rate=1e-5` model for decoding exploration because it achieved the best public BLEU among the trained models.

After applying `no_repeat_ngram_size=2`, the most severe repetition loops were reduced, but many remaining errors were semantic. Some predictions were fluent and loosely related to the source sentence, but they drifted away from the correct meaning. For example, one reference translation was:

inland waterways can be a good theme to base a holiday around

while the model predicted:

The view from the lake is quite nice

This prediction preserved a water-related semantic field, but compressed and shifted the meaning into a generic statement about a lake view.

Another example involved a scientific description of an animal’s plumage. The original Vietnamese sentence was:

các nhà khoa học cho rằng bộ lông của loài này có màu nâu hạt dẻ phía trên và màu xanh xám hoặc vàng/cam bên dưới

The reference translation was:

scientists say this animal’s plumage was chestnut-brown on top with a pale or carotenoid-colored underside

However, the model predicted:

The colors of the wood are brown, green, orange, and orange

This prediction preserved some of the color information from the original sentence, including references to brown, green, and orange tones. However, it completely confused the subject of the sentence, translating the animal’s plumage into “wood,” and lost the scientific description relating the colors to different parts of the animal’s body. The model captured fragments of the underlying meaning but failed to correctly preserve the entities and relationships expressed in the source sentence.

This suggested that greedy decoding with `num_beams=1` might be too weak because it commits to locally likely tokens, while `no_repeat_ngram_size=2` might be too strict because it blocks repeated bigrams even when some repetition is legitimate. Therefore, I tested both beam size and repetition constraint while keeping the trained model fixed.

Table 13. Effect of decoding parameters on the $LR=1 \times 10^{-5}$ model

<code>no_repeat_ngram_size</code>	<code>num_beams</code>	Trainer-style Val. BLEU	Test-style Val. BLEU	Public BLEU
2	1	8.14	8.40	12.56
2	2	7.84	8.85	13.677
2	3	–	9.13	13.841
3	1	7.19	8.61	12.897
3	2	8.02	9.07	14.504
3	3	9.06	9.09	14.455

The decoder ablation shows that test-time decoding had a substantial effect on public BLEU even though the trained model was unchanged. The best result came from combining both changes: `no_repeat_ngram_size=3` and `num_beams=2`, which achieved 14.504 public BLEU.

Importantly, the test-style validation BLEU was more consistent with the public BLEU trend than the original Trainer-style validation BLEU. This supports the decision to use `score_validation_like_test()` as the main local validation criterion for decoding decisions.

However, validation BLEU was still not a perfect predictor of leaderboard performance. This remaining mismatch may come from distribution differences between the FLEURS validation split and the competition test set.

3.4 FLEURS Dataset Influence

To evaluate whether the FLEURS dataset provides useful supervision for the competition task, I compared the performance of the baseline `whisper-small` model before and after LoRA fine-tuning. Both models used the same decoding configuration:

- `num_beams = 2`
- `no_repeat_ngram_size = 3`

The results are shown in Table 15.

Table 14. Effect of FLEURS fine-tuning on `whisper-small`

Model	Public BLEU	Training
<code>whisper-small</code>	14.14	Zero-shot
<code>whisper-small</code>	14.50	LoRA fine-tuned on FLEURS

Fine-tuning on the FLEURS Vietnamese–English speech translation dataset improved public BLEU from 14.14 to 14.50. Although the improvement was modest, it demonstrates that FLEURS contains useful task-relevant supervision and helps adapt the pretrained Whisper model to the Vietnamese–English speech translation task.

3.5 Model Capacity

Error analysis revealed that many low-BLEU predictions still suffered from semantic and acoustic failures even when the decoder produced fluent outputs.

Several examples showed failures in recognizing named entities. For instance, references containing political figures such as *Newt Gingrich* and *Michele Bachmann* were translated as "New England", "Mitchell Batman". Similarly, technical terms and abbreviations such as *APS* were incorrectly translated into semantically unrelated phrases such as "folk songs" or "ancient legends". These errors appeared to originate from limitations in acoustic understanding, semantic representation, and the handling of rare or specialized vocabulary.

This observation motivated an investigation into model capacity. To test this hypothesis, I evaluated a zero-shot `whisper-medium` and `whisper-large-v3` model without any fine-tuning while keeping the same decoding configuration:

- `num_beams = 2`
- `no_repeat_ngram_size = 3`

The result is shown in Table 15.

Table 15. Comparison among model capacity

Model	Public BLEU	Training
Best <code>whisper-small</code> LoRA	14.50	Fine-tuned
<code>whisper-medium</code>	23.23	Zero-shot
<code>whisper-medium</code>	23.60	Fine-tuned
<code>whisper-large-v3</code>	25.55	Zero-shot

The results show a strong relationship between model capacity and translation performance. Even without any task-specific fine-tuning, the zero-shot `whisper-medium` model achieved a public BLEU score of 23.23. The trend continued with `whisper-large-v3`, which achieved the highest public BLEU score of 25.55 without any fine-tuning. Notably, this zero-shot model outperformed the fine-tuned `whisper-medium` model by nearly 2

BLEU points and exceeded the best fine-tuned `whisper-small` model by more than 11 BLEU points.

These improvements are substantially larger than the gains obtained from decoder tuning or FLEURS fine-tuning alone. Therefore, increasing model capacity emerged as the most effective strategy explored in this study.

4. Conclusion

The two strongest submissions obtained during this study are summarized below.

Submission A: Fine-Tuned Whisper-Medium (Public BLEU: 23.60)

- Base model: `openai/whisper-medium`
- Language: `vi`
- Task: `translate`
- Fine-tuning method: LoRA
- Dataset: FLEURS Vietnamese–English
- Learning rate: 1×10^{-5}
- Number of epochs: 3
- Per-device batch size: 8
- Gradient accumulation: 4
- Effective batch size: 32
- LoRA rank (r): 8
- LoRA alpha (α): 16
- LoRA dropout: 0.05
- Target modules: `q_proj`, `k_proj`, `v_proj`, `out_proj`, `fc1`, `fc2`
- Decoder: beam search
- `num_beams` = 2
- `no_repeat_ngram_size` = 3

Submission B: Zero-Shot Whisper-Large-v3 (Public BLEU: 25.55)

- Base model: `openai/whisper-large-v3`
- Language: `vi`
- Task: `translate`
- Fine-tuning: none
- Decoder: beam search
- `num_beams` = 2
- `no_repeat_ngram_size` = 3

Overall, the experiments suggest that increasing model capacity produced substantially larger gains than decoder tuning or fine-tuning alone. While FLEURS fine-tuning consistently improved performance, the largest improvements came from moving from `whisper-small` to `whisper-medium` and ultimately to `whisper-large-v3`.